

# WebAR公開手順

---

テンプレートを自分のGitHubリポジトリへコピー  
↓  
Unityでスタンプ取得処理を制作  
↓  
WebGLとしてdocsフォルダーへビルド  
↓  
GitHubへPush  
↓  
GitHub Pagesで公開  
↓  
スマートフォンでカメラ・マーカ認識を確認

---

## 準備

- GitHubアカウント
- GitHub Desktop
- Unity Hub
- Unity 6 [6000.3.14f1](#)
- UnityのWeb Build Support

---

## リポジトリ作成

自分用のリポジトリを作成

Publicリポジトリで作成

---

## GitHub Desktopでプロジェクトを取得

1. 自分のリポジトリで [Code](#) を押す。
2. [Open with GitHub Desktop](#) を選ぶ。
3. 保存先を確認して [Clone](#) を押す。
4. Cloneが完了したら、GitHub Desktopの [Show in Explorer](#) でフォルダーを確認する。

Assets/  
Packages/  
ProjectSettings/

---

## Unityでプロジェクトを新規作成

作成したリポジトリフォルダにプロジェクトを作成

Universal 2Dプロジェクト

---

## UnityPackageをインポート

<https://www.is.kyusan-u.ac.jp/~sumida/class/tools/webartemplate.unitypackage>

## サンプルシーン

```
Assets/WebARStamp/Samples/WebARSample.unity
```

### サンプルシーンの動作

1. WebARを起動ボタン
2. 対応するマーカを読み込むと、Textをマーカ名に変更
3. ボタンの色を変更

Unity Editorでエラーが出ないことを確認

Consoleに次のメッセージが表示されることを確認する。

```
WebARはWebGLビルドで実行してください。
```

---

## AR読み取り結果を受け取るスクリプト

```
Assets/WebARStamp/Runtime/WebARSample.cs  
Assets/WebARStamp/Samples/WebARSample.unity
```

次のファイルはWebARの内部処理。基本的には変更しない。

```
Assets/WebARStamp/Runtime/WebARBridge.cs  
Assets/Plugins/WebGL/WebARBridge.jslib  
Assets/WebGLTemplates/WebARStamp/
```

---

## 認識結果を受け取る

マーカを認識すると、WebARSample.csのメソッドが呼び出される。

```
public void OnMarkerDetected(string markerName)
{
    Debug.Log("読み取り結果: " + markerName);
}
```

`markerName`には次のいずれかが入る。（マーカーは配布時は3種類）

| 認識したマーカー | <code>markerName</code> |
|----------|-------------------------|
| 1枚目      | <code>spot_01</code>    |
| 2枚目      | <code>spot_02</code>    |
| 3枚目      | <code>spot_03</code>    |

```
public void OnMarkerDetected(string markerName)
{
    if (markerName == "spot_01")
    {
        // スタンプ1を取得したときの処理
    }
    else if (markerName == "spot_02")
    {
        // スタンプ2を取得したときの処理
    }
    else if (markerName == "spot_03")
    {
        // スタンプ3を取得したときの処理
    }
}
```

---

---

## GitHub Pages向けのWebGL設定

### Build Profileを確認する

1. **File** → **Build Profiles** を開く。
2. **Web** を選ぶ。
3. Web向けになっていない場合は **Switch Platform** を押す。
4. Scene Listに次のシーンが登録され、チェックが付いていることを確認する。

```
Assets/WebARStamp/Samples/WebARSample.unity
```

### 圧縮設定を変更する

**Player** → **Web** → **Publishing Settings** を開き、次のように設定。

Compression Format: Brotli  
Decompression Fallback: ON

## docs フォルダへWebGLビルドする

1. **File** → **Build Profiles** を開く。
2. **Web** が選択されていることを確認する。
3. **Build** を押す。
4. 保存先として、プロジェクト直下の **docs** フォルダを指定する。

```
プロジェクトフォルダー/  
├ Assets/  
├ Packages/  
├ ProjectSettings/  
└ docs/ ← ここへBuild
```

**docs**が存在しない場合は、保存画面で新しく作成。

ビルド完了後、次のようなファイルが生成されていることを確認。

```
docs/  
├ index.html  
├ style.css  
├ webar.js  
├ targets.mind  
├ Build/  
├ Libraries/  
└ Overlays/
```

## GitHubへPush

1. GitHub Desktopを開く。
2. 左側のChangesに、自分の編集と**docs**内のビルド結果が表示されることを確認する。
3. **Summary**へ変更内容を入力して commit、push。
4. GitHub Webの自分のリポジトリを開き、**docs/index.html**が存在することを確認する。

## GitHub Pagesを有効にする

この設定は、リポジトリごとに最初の1回だけ行う。

1. GitHubで自分のリポジトリを開く。
2. **Settings** を開く。

3. 左側の **Pages** を選ぶ。
4. **Build and deployment**の**Source**を次に設定する。

```
Deploy from a branch
```

5. Branchとフォルダーを次のように設定する。

```
Branch: main  
Folder: /docs
```

6. **Save** を押す。
7. 公開処理が終わるまで待つ。

GitHubの **Settings** → **Pages** に表示されたURLを開いて動作確認。

---

## PCブラウザ、スマートフォンで確認

公開URLをPCのChromeなどで開く。WebARを起動ボタン、マーカを読み込む。マーカを認識したら元の画面に戻り、読み込んだマーカの名前が表示されることを確認。